

Boolean Rule Language Compiler

This project is a hand-built compiler front end and small interpreter for a custom boolean rule language.

The language uses a custom surface syntax (emoji-inspired operators), while still implementing the required boolean/comparison/arithmetic parsing behavior.

Implemented features:

- Boolean literals
- Identifiers
- Logical operators (`and`, `or`, `not`) via custom tokens
- Comparison operators (`<`, `>`, `=`, `!=`, `<=`, `>=`, `<`, `>`)
- Arithmetic subexpressions inside comparisons
- Assignment statements
- Output statements
- AST construction for arithmetic and boolean expressions
- Syntax error reporting with line/column positions
- Comment support
- Simplified interpreter behavior

Custom Syntax Mapping:

- `:=` -> assignment (`:=` equivalent)
- `~` -> statement terminator (`;` equivalent)
- `leviosa` -> print keyword (`print` equivalent)
- `:)` -> `true`
- `:(` -> `false`
- `{` -> logical `and`
- `[` -> logical `or`
- `!` -> logical `not`
- `>` expr -<` -> grouped expression (`(expr)` equivalent)
- `` `comment` `` -> comment block

Token Specification:

- Literals and names: `IDENTIFIER`, `NUMBER`, `TRUE`, `FALSE`
- Statements: `PRINT`, `ASSIGN`, `END`
- Arithmetic: `PLUS`, `MINUS`, `STAR`, `SLASH`
- Boolean/logical: `AND`, `OR`, `NOT`
- Comparison: `EQUAL`, `NOT\_EQUAL`, `LESS`, `LESS\_EQUAL`, `GREATER`, `GREATER\_EQUAL`
- Grouping: `LPAREN`, `RPAREN`
- Stream end: `EOF`

Each token stores:

- token type
- lexeme (raw source slice)
- optional literal value (`Integer` or `Boolean`)
- source position (line and column)

Grammar Description:

Grammar used by recursive descent parser (`parser/Parser.java`):

```
program    -> statement* EOF ;
statement  -> assignment | print ;
assignment -> IDENTIFIER ASSIGN expr END ;
print      -> PRINT expr END ;
expr       -> or ;
or         -> and (OR and)* ;
and        -> comparison (AND comparison)* ;
comparison -> not ((< | <= | > | >= | = | !=) not)? ;
not        -> NOT not | arithmetic ;
arithmetic -> term ((+ | -) term)* ;
term       -> factor ((* | /) factor)* ;
factor     -> NUMBER | TRUE | FALSE | IDENTIFIER | LPAREN expr RPAREN ;
```

Notes:

- Precedence is encoded by parser function layering.
- Chained comparisons are rejected intentionally.
- Parser uses recovery (`synchronize`) after statement-level failures.

Token demo.java2 visualization:

TYPE	LEXEME	LITERAL	POSITION
IDENTIFIER	'total'		1:1
ASSIGN	':-'		1:7
NUMBER	'1'	1	1:11
PLUS	'+'		1:13
NUMBER	'23'	23	1:15

PLUS	'+'		1:18
NUMBER	'24'	24	1:20
STAR	'*'		1:24
NUMBER	'35'	35	1:26
END	'~'		1:29
IDENTIFIER	'ok'		3:1
ASSIGN	'-:'		3:4
TRUE	' :)'	true	3:7
AND	'{'		3:10
NOT	'!'		3:13
FALSE	' ('	false	3:16
END	'~'		3:18
PRINT	'leviosa'		4:1
LPAREN	'->'		4:9
IDENTIFIER	'total'		4:12
RPAREN	'-<'		4:18
END	'~'		4:20
PRINT	'leviosa'		5:1
LPAREN	'->'		5:9
IDENTIFIER	'ok'		5:11
RPAREN	'-<'		5:13
END	'~'		5:15
EOF	'"		6:1

Scanner and Parser Diagnostics:

- Lexical errors: thrown as `LexicalException` with source position.
- Syntax errors: collected as `ParseException` with source position.
- Runtime type/evaluation errors: thrown as `EvalException`.

Scanner-only visibility:

AST Design

Root and statement nodes:

- `Program`
- `AssignmentStmt`
- `PrintStmt`

Expression nodes:

- `BinaryExpr`
- `UnaryExpr`
- `LiteralExpr`
- `IdentifierExpr`
- `GroupExpr`

All nodes carry source position information.

AST Output Examples (Structural)

Input:

```
adult :- age >= 18 ~  
leviosa adult ~
```

Example structure:

```
Program  
  AssignmentStmt(name=adult)  
    BinaryExpr(op=>=)  
      IdentifierExpr(age)  
      LiteralExpr(18)  
  PrintStmt  
    IdentifierExpr(adult)
```

Input:

```
approved :- income > 5000 :{ :! blocked ~
```

Example structure:

```
Program  
  AssignmentStmt(name=approved)  
    BinaryExpr(op=AND)  
      BinaryExpr(op=>)
```

IdentifierExpr(income)
LiteralExpr(5000)
UnaryExpr(op=NOT)
IdentifierExpr(blocked)

AST demo.java2 visualization:

--- ABSTRACT SYNTAX TREE ---

Program

Assignment: total

Value: Binary(+)

Left: Binary(+)

Left: Literal[1]

Right: Literal[23]

Right: Binary(*)

Left: Literal[24]

Right: Literal[35]

Assignment: ok

Value: Binary(:{)}

Left: Literal[true]

Right: Unary(:!)

Right: Literal[false]

Print (leviosa)

Expr: Grouping

Inside: Id[total]

Print (leviosa)

Expr: Grouping

Inside: Id[ok]

Test Cases

- Valid: nontrivial arithmetic/boolean/grouping combinations
- Invalid: lexer failures, parser failures, chained comparison rejection

Run tests:

```
javac Main.java lexer/*.java parser/*.java ast/*.java util/*.java runtime/*.java  
tests/TestRunner.java  
java tests.TestRunner
```

Demo:

Run program:

```
java Main demo.java2
```

Run with token stream output:

```
java Main -t demo.java2
```

This demonstrates scanner output, parser integration, and interpreter output in one flow.

```
java Main -t -a demo.java2
```

This demonstrates scanner output, parser integration, interpreter, and AST output in one flow.

Short Design Report

Design decisions:

1. Recursive descent parser was chosen for direct grammar-to-code mapping and control over precedence.
2. Lexer stores both lexeme and optional typed literal value for clearer downstream semantics.
3. AST separates syntax from runtime evaluation to keep parser clean and testable.
4. Position-aware exceptions provide actionable diagnostics for both lexical and parse phases.
5. Comparison parsing is intentionally non-chainable to enforce explicit boolean expressions.